# Trees

Jianguo Lu

February 5, 2025

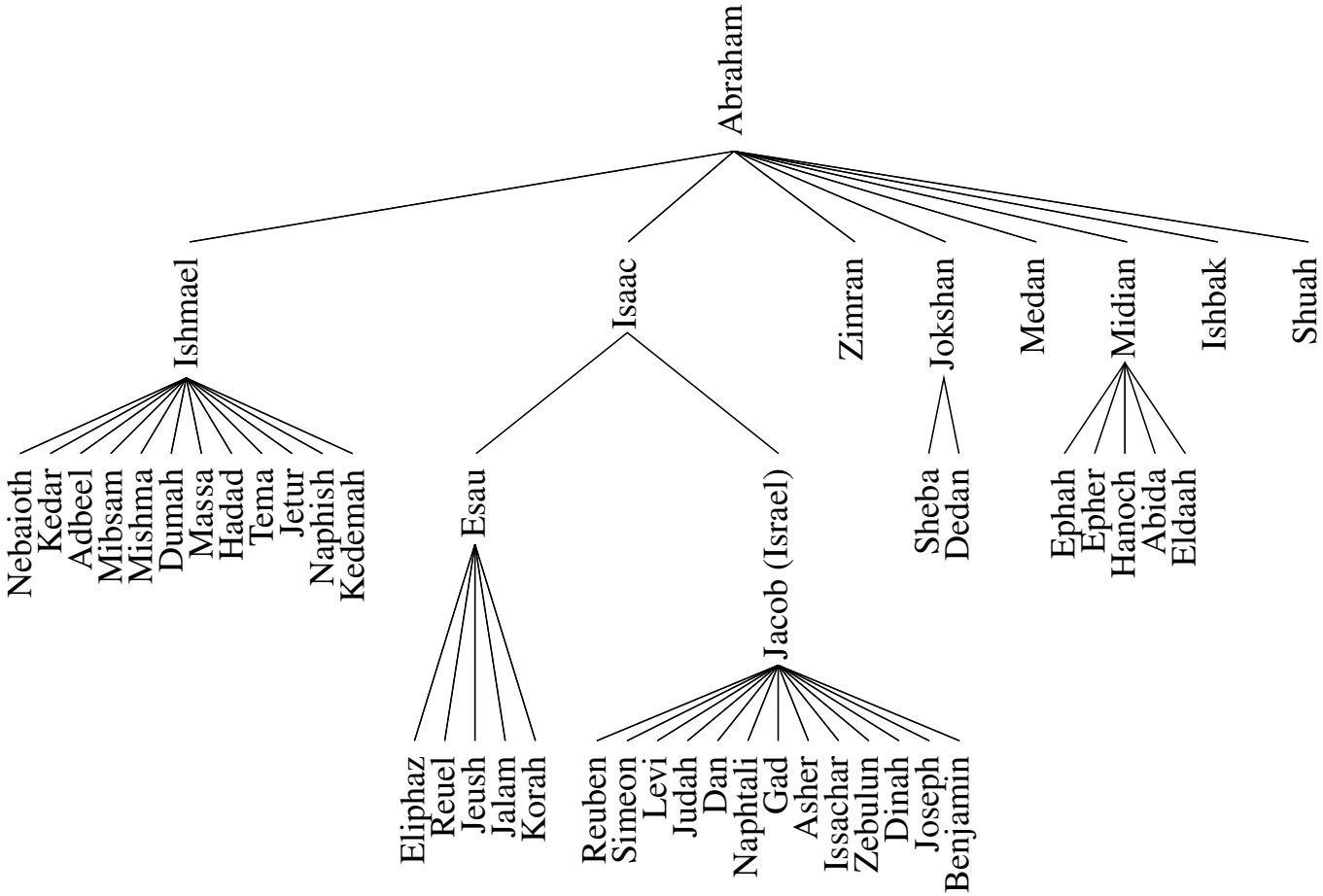
# Overview

# Why do we need trees?

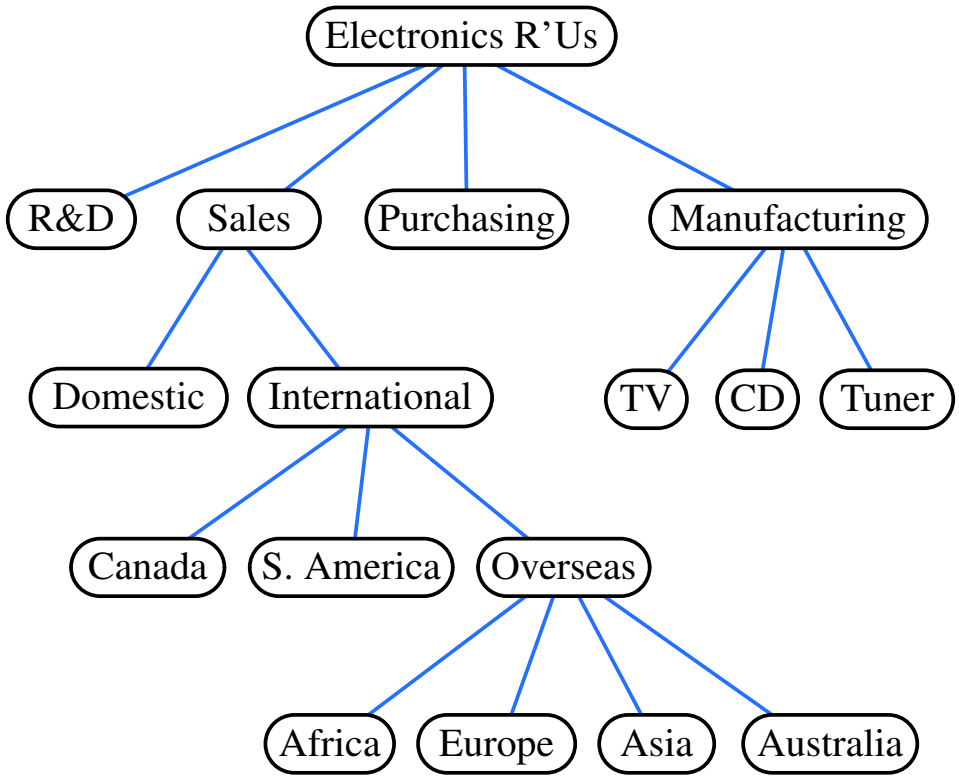
- Stack, Queue, List are linear data structures
- Information often contain hierarchical relationships
  - Hierarchical organizations
  - File directories
  - Moves in game, decision trees
  - Intermediate result in compilers
  - ...

# Tree Examples

Family tree

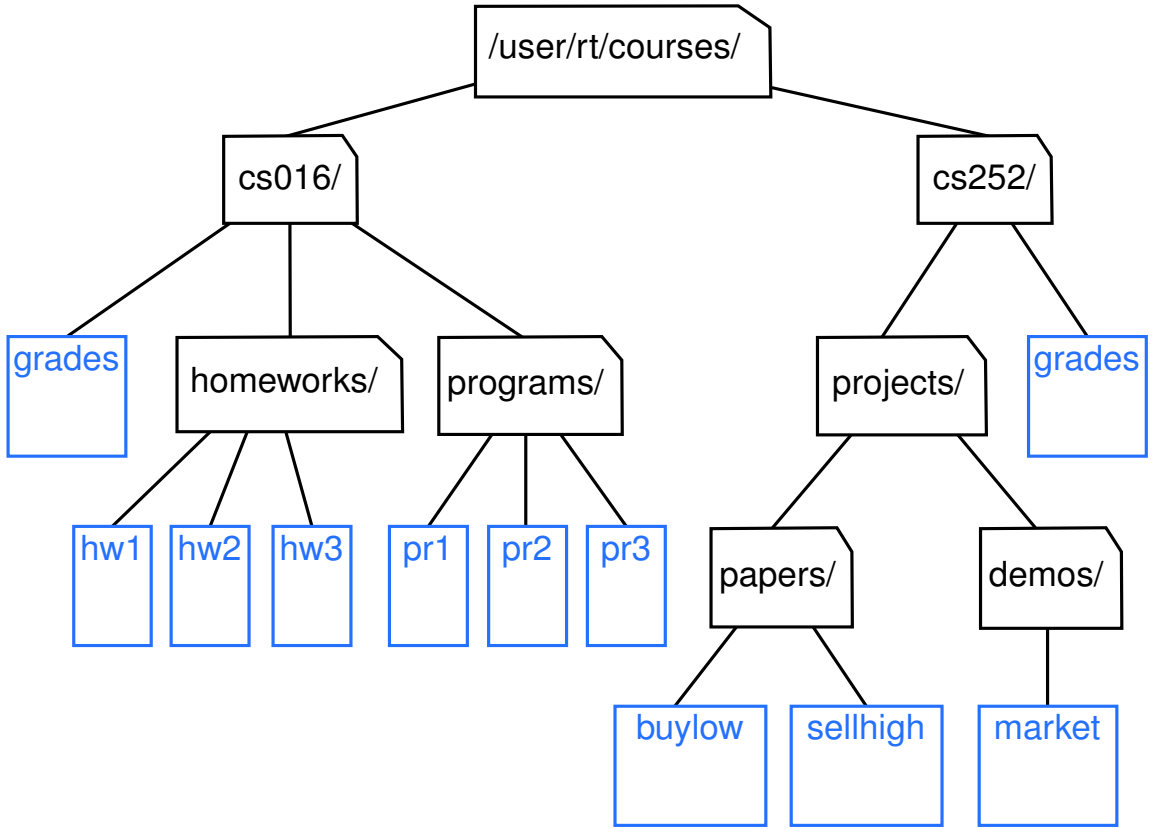


# Tree examples



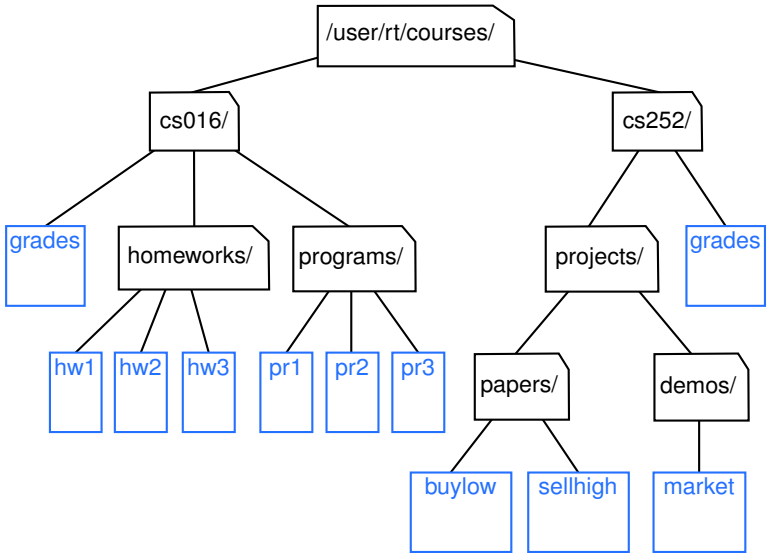

Corporation organization

# Tree example




File Directories

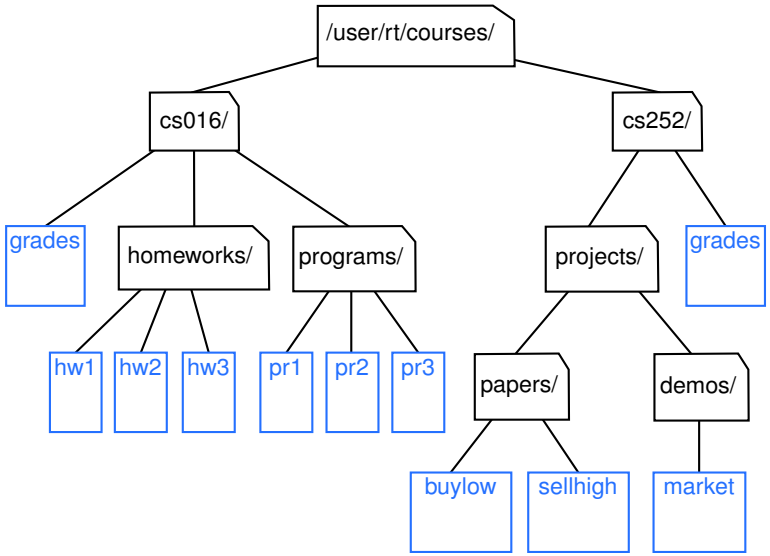
# Formal definition of trees




## Definition of a tree

- A set of nodes.

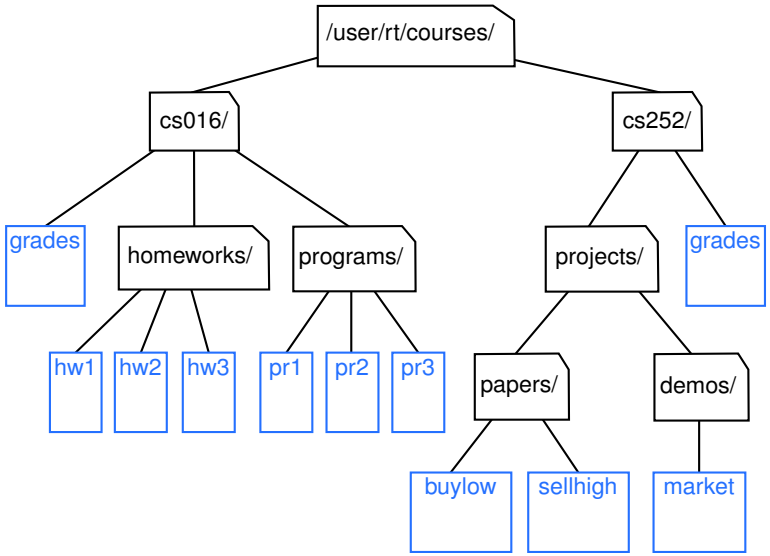
# Formal definition of trees




## Definition of a tree

- A set of nodes.
- Nodes have parent-child relationship

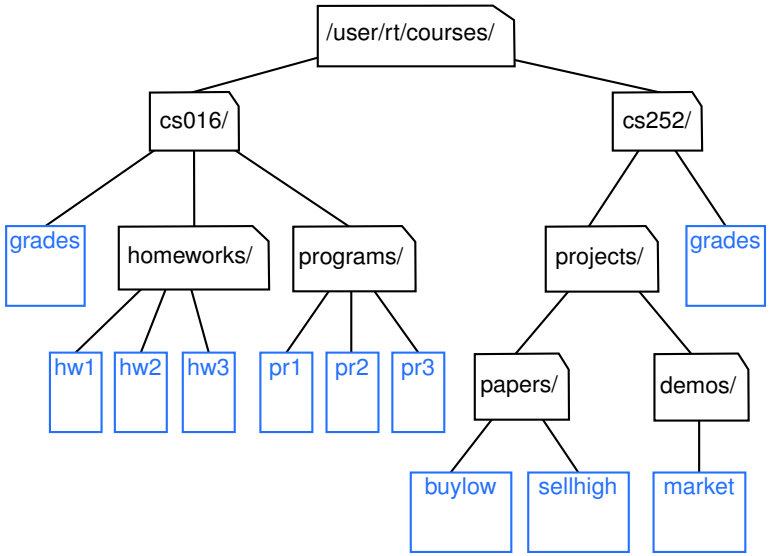
# Formal definition of trees




## Definition of a tree

- A set of nodes.
- Nodes have parent-child relationship
- It has a special node called the root of the tree that has no parent

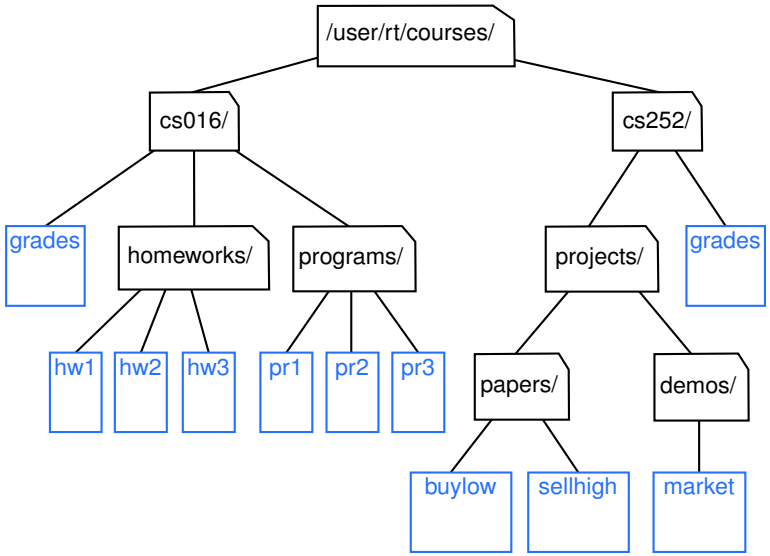
# Formal definition of trees




## Definition of a tree

- A set of nodes.
- Nodes have parent-child relationship
- It has a special node called the root of the tree that has no parent
- Each other node has a unique parent

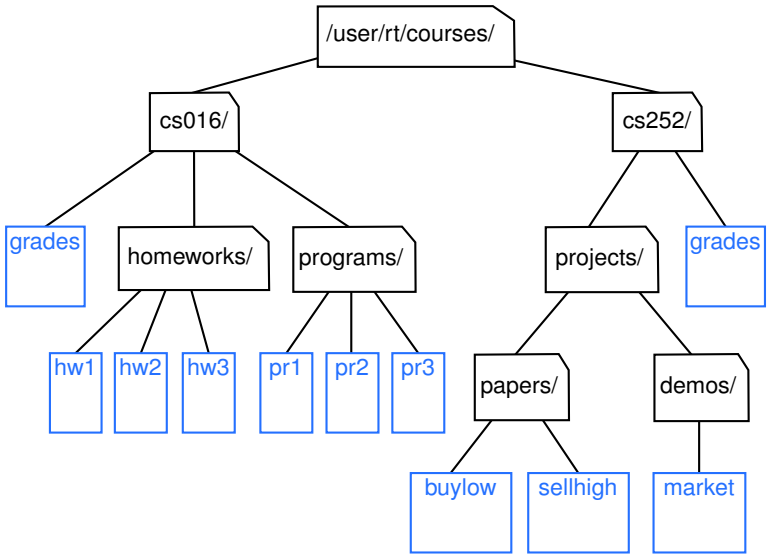
# Formal definition of trees




## Definition of a tree

- A set of nodes.
- Nodes have parent-child relationship
- It has a special node called the root of the tree that has no parent
- Each other node has a unique parent

# Tree Jargon




- Sibling
- leaf
- Depth of a node N: path length from root to N
- Height of a node: length of the longest path from N to a leaf

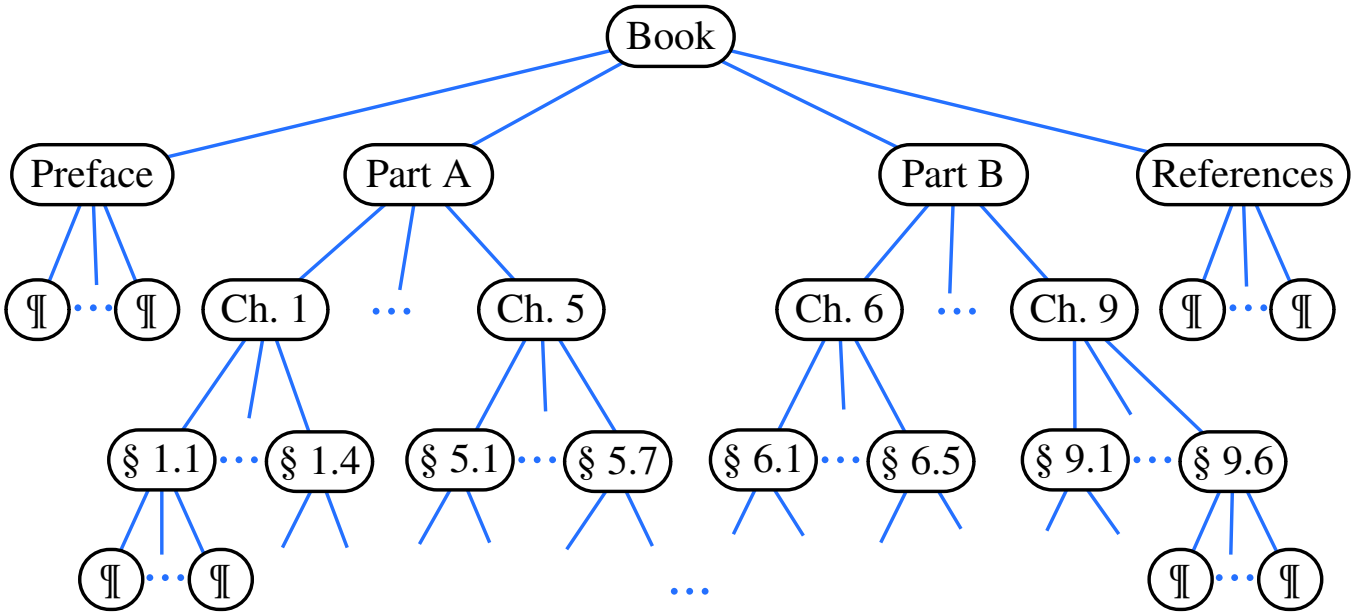
# Number of nodes and edges

**Property**

A tree with N nodes always has N-1 edges

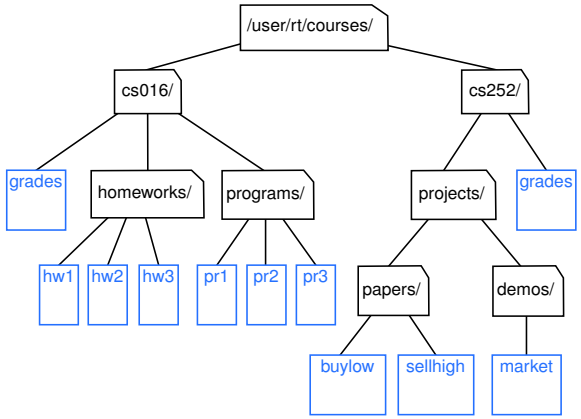
Prove: induction

# Ordered trees




The order of the siblings are relevant

# Depth of a node




## Depth of a node

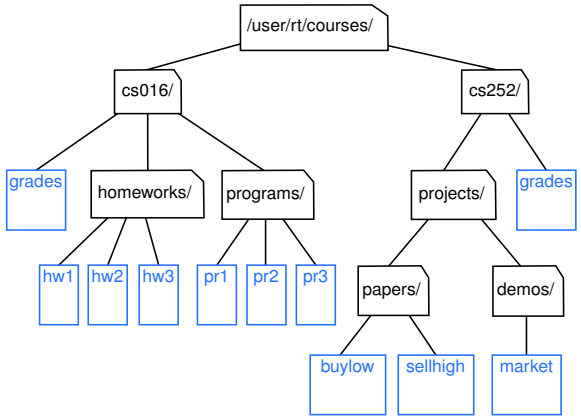
The depth of *node* is the number of ancestors of *node*, other than *node* itself.

The depth of a node can be recursively defined as:

- If *node* is the root, then the depth of *node* is 0.

```
public int depth(Node node) {
    if (isRoot(node))
      return 0;
    else
      return 1 + depth(parent(node));
  }
```

# Depth of a node




## Depth of a node

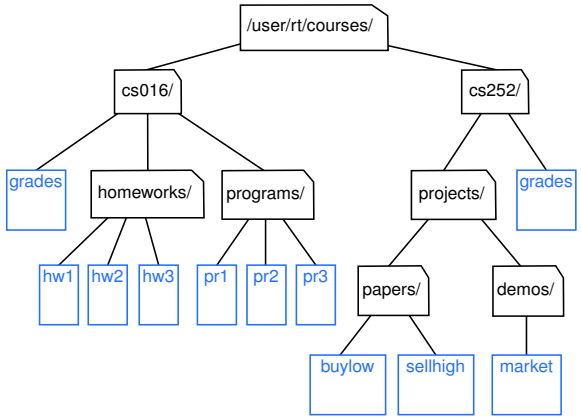
The depth of *node* is the number of ancestors of *node*, other than *node* itself.

The depth of a node can be recursively defined as:

- If *node* is the root, then the depth of *node* is 0.
- Otherwise, the depth of *node* is one plus the depth of the parent of *node*.

```
public int depth(Node node) {
    if (isRoot(node))
      return 0;
    else
      return 1 + depth(parent(node));
  }
```

# Depth of a node




## Depth of a node

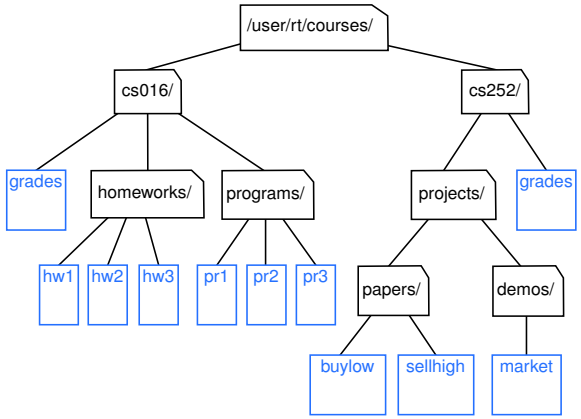
The depth of *node* is the number of ancestors of *node*, other than *node* itself.

The depth of a node can be recursively defined as:

- If *node* is the root, then the depth of *node* is 0.
- Otherwise, the depth of *node* is one plus the depth of the parent of *node*.

```
public int depth(Node node) {
    if (isRoot(node))
      return 0;
    else
      return 1 + depth(parent(node));
  }
```

# Depth of a node




## Depth of a node

The depth of *node* is the number of ancestors of *node*, other than *node* itself.

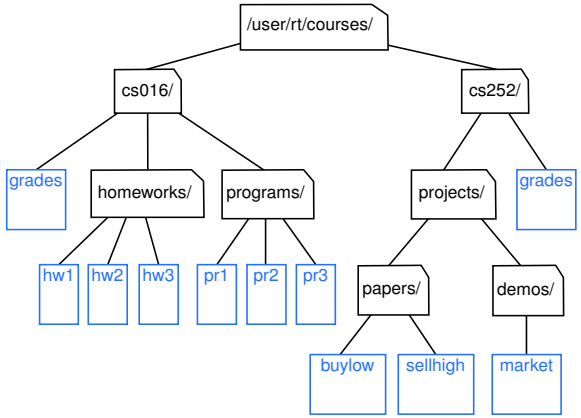
The depth of a node can be recursively defined as:

- If *node* is the root, then the depth of *node* is 0.
- Otherwise, the depth of *node* is one plus the depth of the parent of *node*.

```
public int depth(Node node) {
    if (isRoot(node))
      return 0;
    else
      return 1 + depth(parent(node));
  }
```

- The complexity of the algorithm?

# Height of a tree



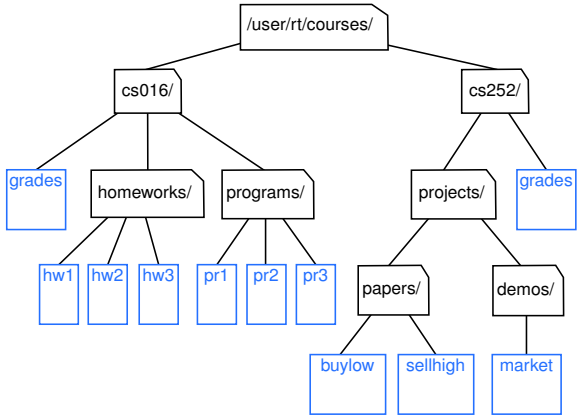

## Definition of Height

The height of a tree is the maximum depth of all the nodes in a tree

```
private int heightBad( ) {
        int h = 0;
        for (Node node : nodes( ))
                if (isExternal(node))
                        h = Math.max(h, depth(node));
        return h;
}
```

- The complexity of the algorithm?

# Height of a tree




## Definition of Height

The height of a tree is the maximum depth of all the nodes in a tree

```
private int heightBad( ) {
        int h = 0;
        for (Node node : nodes( ))
                if (isExternal(node))
                        h = Math.max(h, depth(node));
        return h;
}
```

- The complexity of the algorithm?
- The worst case time complexity of $O(n^2)$.

# Height of a tree: A better algorithm

- If *node* is a leaf, then the height of *node* is 0.
- Otherwise, the height of node is one more than the maximum of the heights of its children.

```
public int height(Node node) {
    int h = 0;
    for (Node c: children(node))
      h = Math.max(h, 1 + height(c));
    return h;
  }
```

- What is the complexity of the algorithm?

# Height of a tree: A better algorithm

- If *node* is a leaf, then the height of *node* is 0.
- Otherwise, the height of node is one more than the maximum of the heights of its children.

```
public int height(Node node) {
    int h = 0;
    for (Node c: children(node))
      h = Math.max(h, 1 + height(c));
    return h;
  }
```

- What is the complexity of the algorithm?
- $O(n)$

# Binary Tree

## Definition: Binary Tree

A binary tree is an ordered tree with the following properties:

- Each node has at most two children;
- Each child is labeled as either a left child or a right child
- A left child precede a right child

What if there is one child?

# Binary Tree

## Definition: Binary Tree

A binary tree is an ordered tree with the following properties:

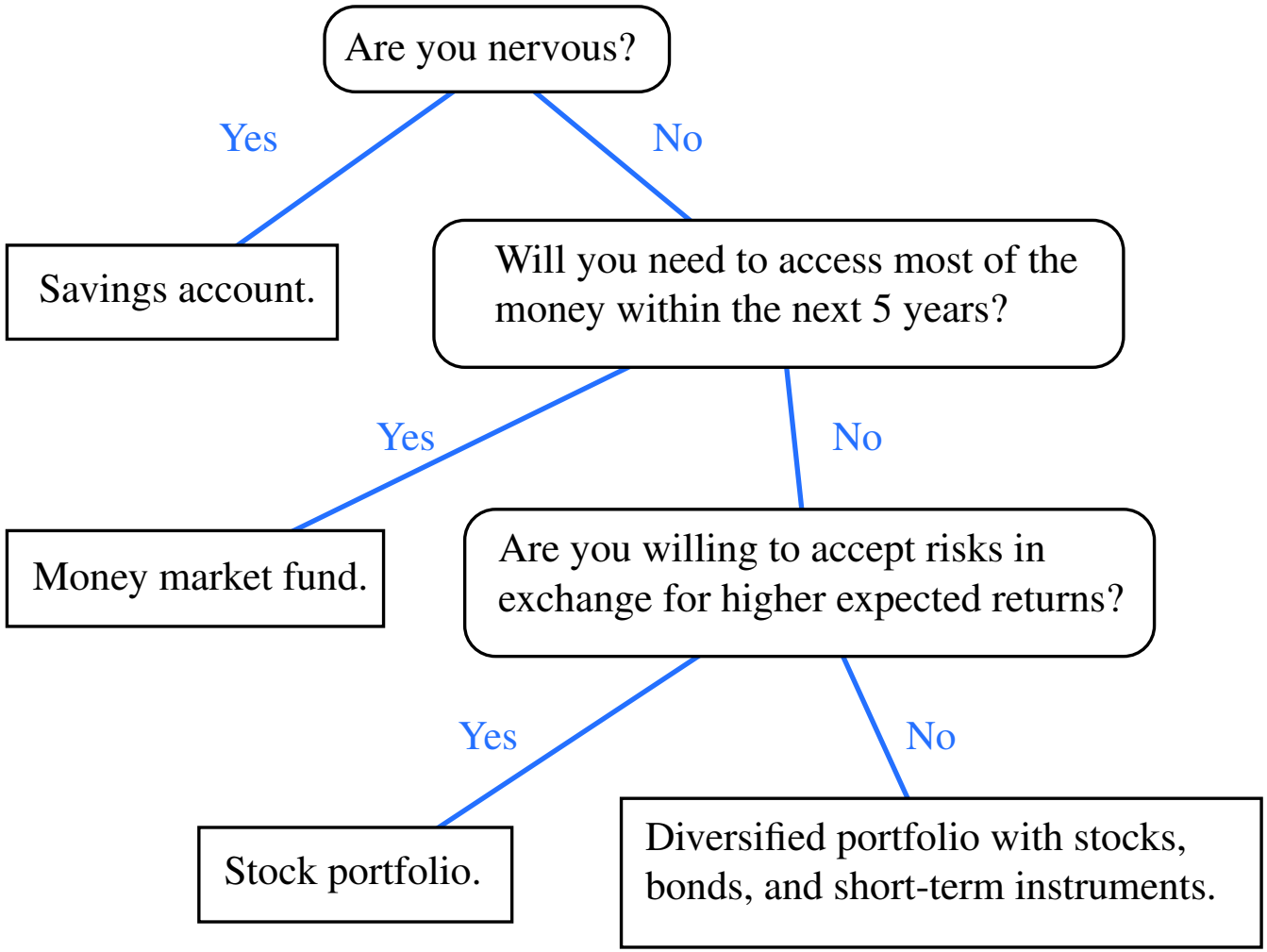
- Each node has at most two children;
- Each child is labeled as either a left child or a right child
- A left child precede a right child

What if there is one child?

## Proper Binary Tree

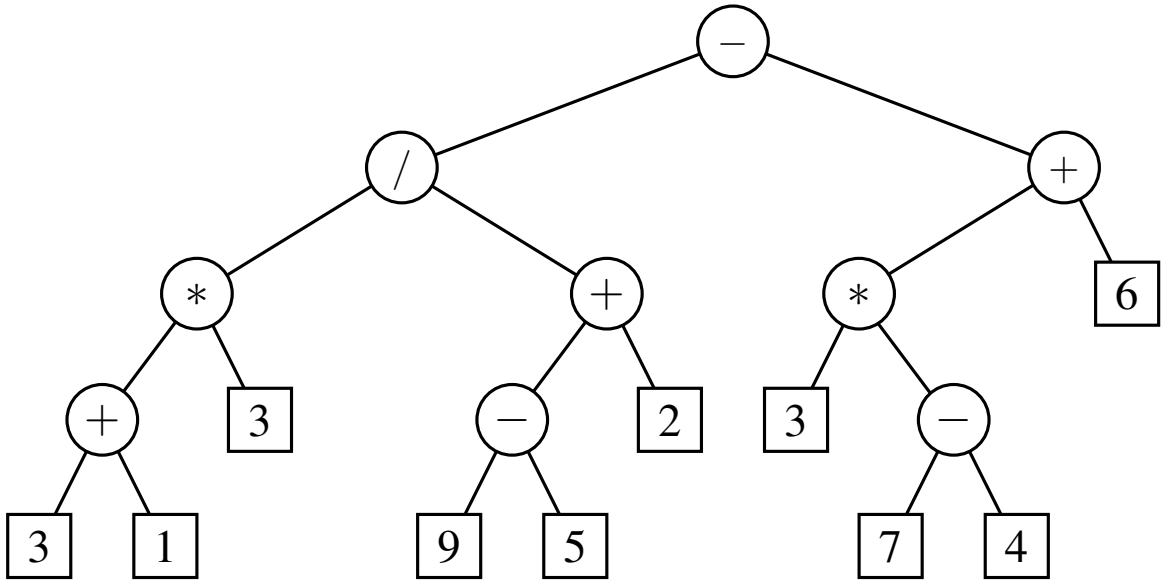
- Each node has either 0 or 2 children
- Proper binary tree = full binary tree

# Binary tree example: Decision tree




Decision tree

# Arithmetic expression

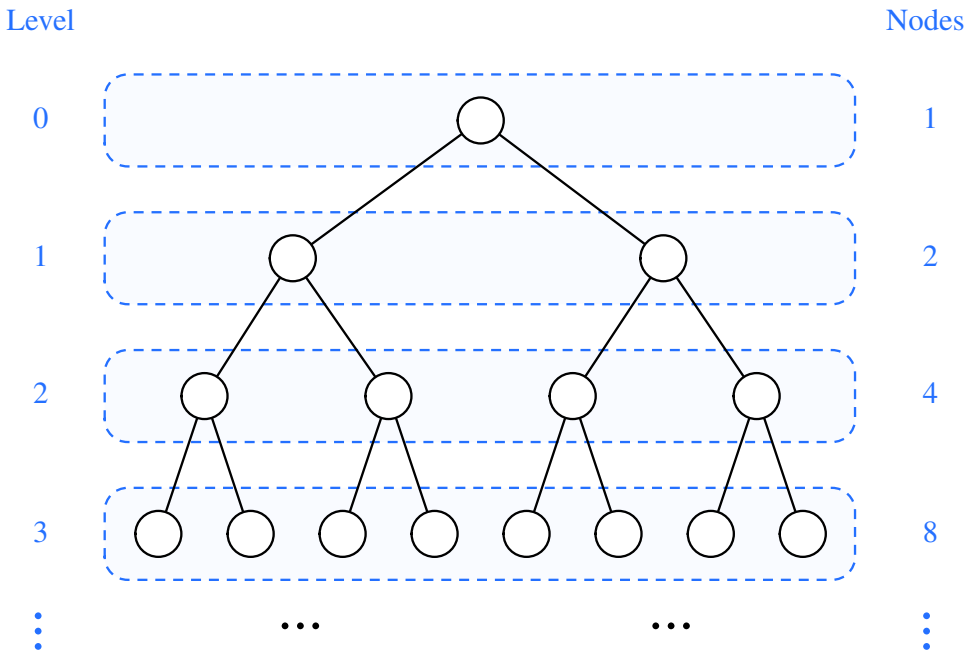



Arithmetic expressions

# Properties of Binary trees: height $h$ and node size $n$

Given $n$ nodes,

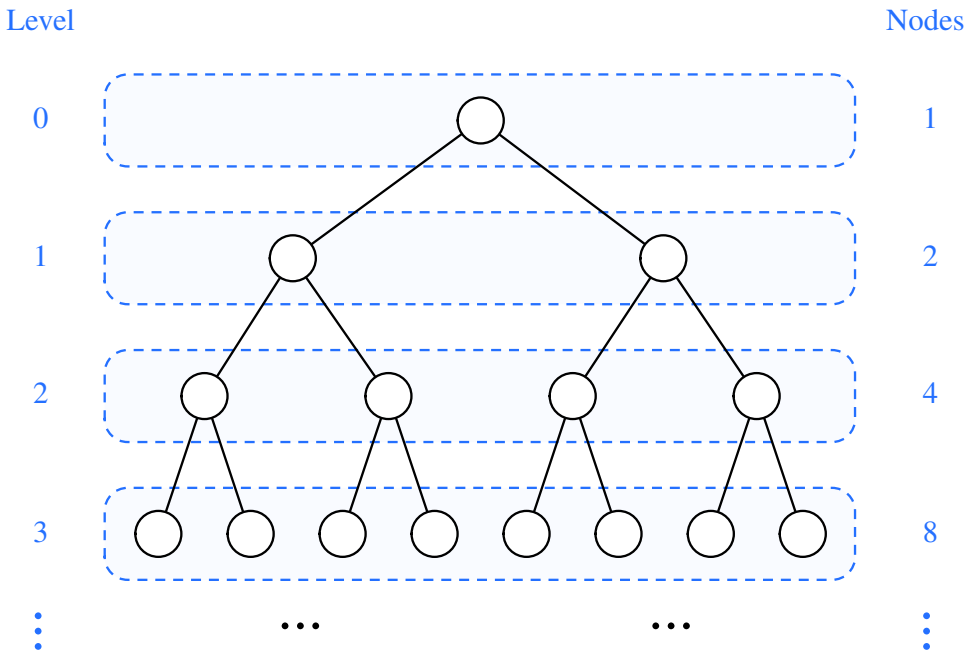
- What is the maximum height of a binary tree?



# Properties of Binary trees: height $h$ and node size $n$

Given $n$ nodes,

- What is the maximum height of a binary tree?
- What is the minimum height of the tree?



# Properties of Binary trees: height $h$ and node size $n$

**Proposition 8.7 (h and n) for any binary tree**

$$h + 1 \leq n \leq 2^{h+1} - 1 \tag{1}$$

**Proposition 8.7 (h and n) for proper binary tree**

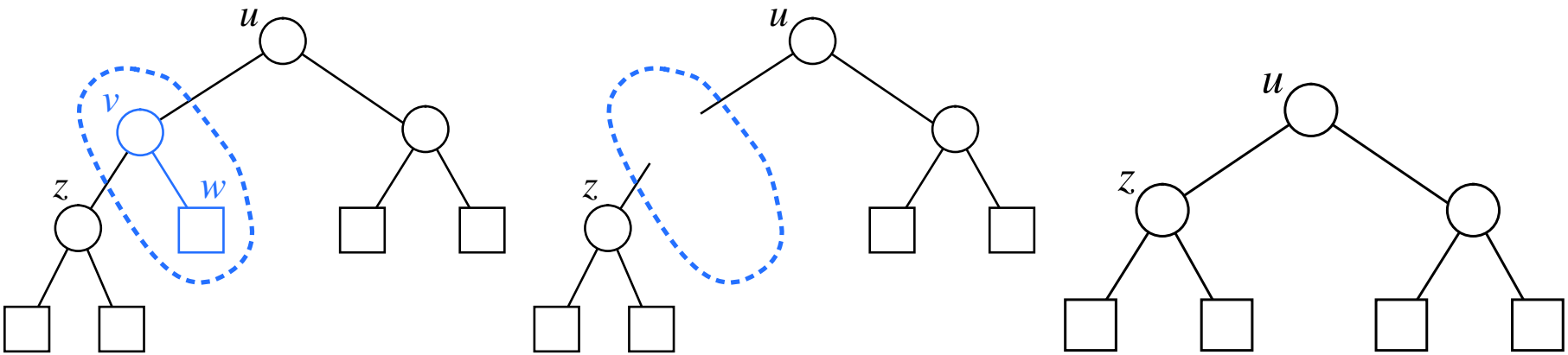
$$2h + 1 \leq n \leq 2^{h+1} - 1 \tag{2}$$

# Properties of Binary trees: external and internal nodes

## Proposition 8.8

In a *proper* binary tree with $n_E$ external nodes and $n_I$ internal nodes, we have

$$n_E = n_I + 1 \tag{3}$$



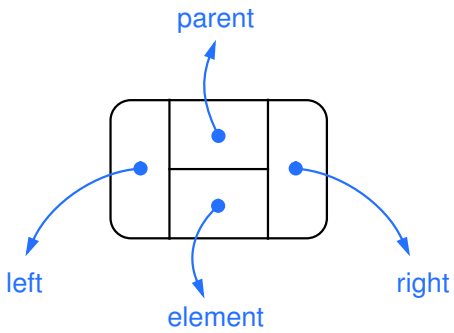
# Implementing trees

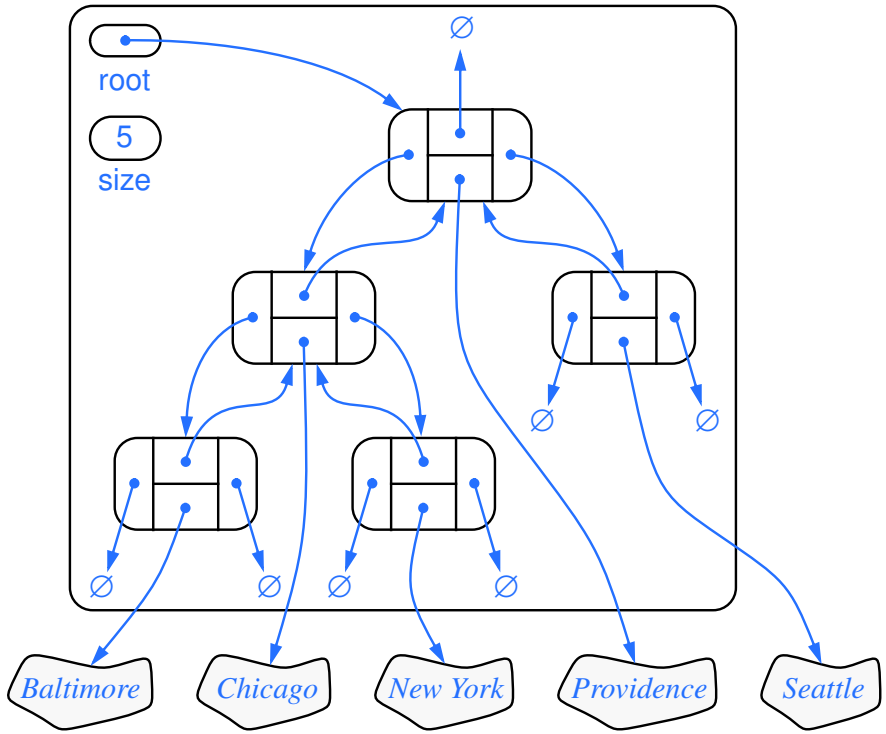
- Nodes and links

# Implementing trees

- Nodes and links
- Arrays

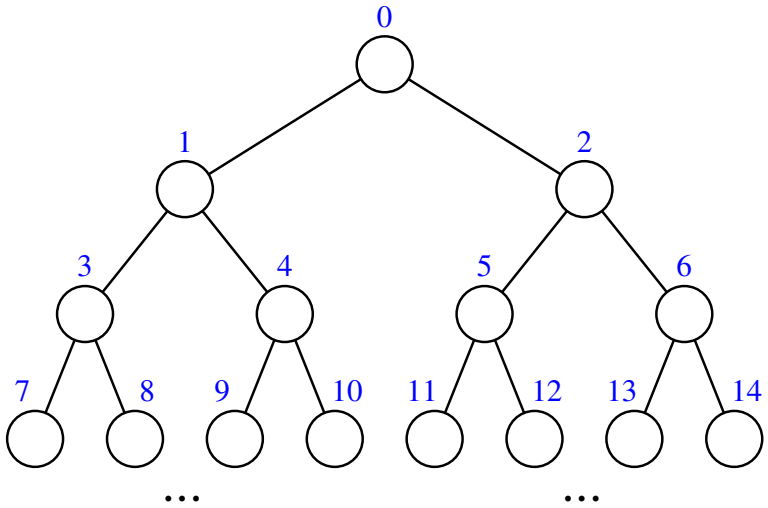
# Implementing trees using links–Binary tree

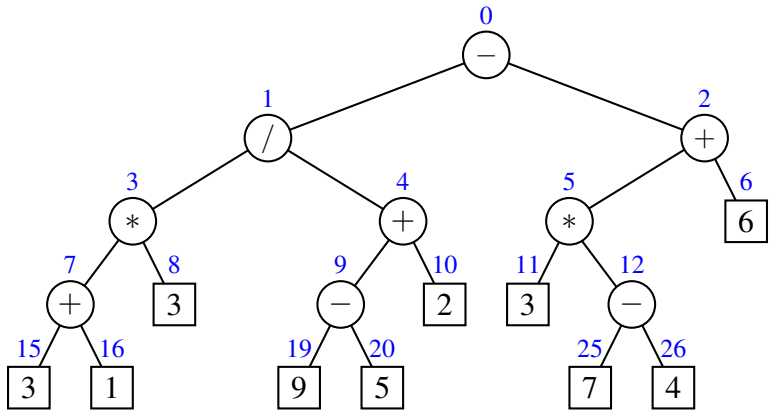


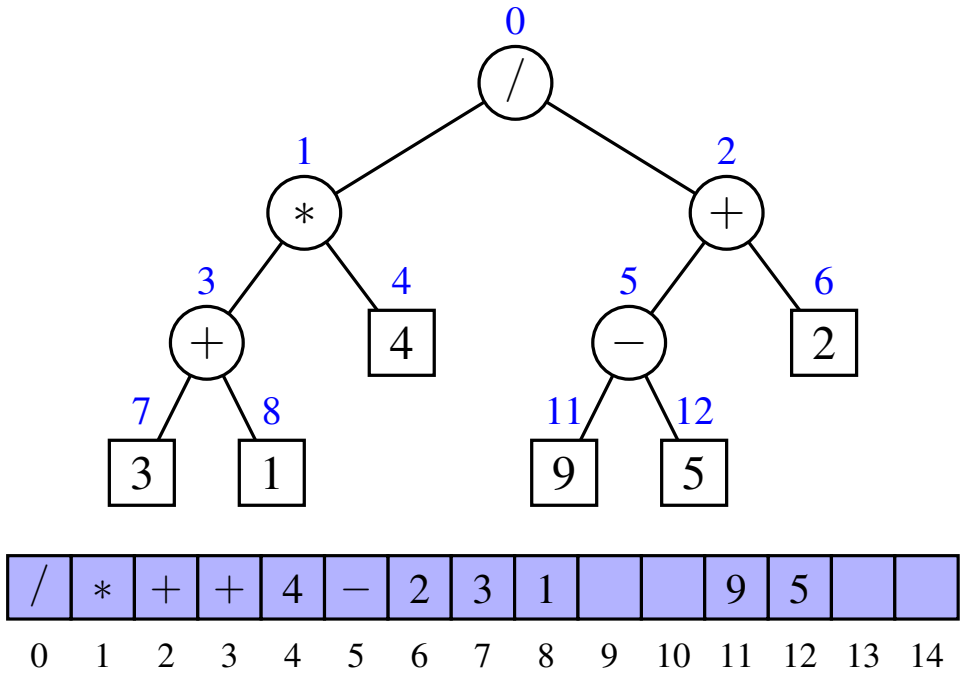

```
protected static class Node<E>  {
    private E element;
    private Node<E> parent;
    private Node<E> left;
    private Node<E> right;
```

# Array based representation



# Array based representation




The index for the node in i-th layer and j-th element is:

$$2^{i-1} - 1 + j \tag{4}$$

The index for node '-' is $2^2 - 1 + 2 = 5$
The index for node '9' is $2^3 - 1 + 4 = 11$

# Array based representation: pros and cons

- save space

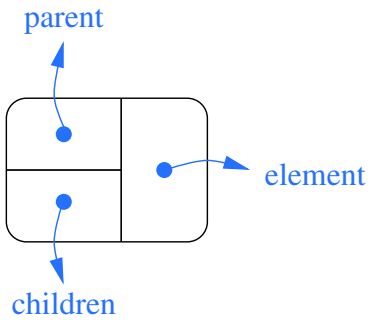
# Array based representation: pros and cons

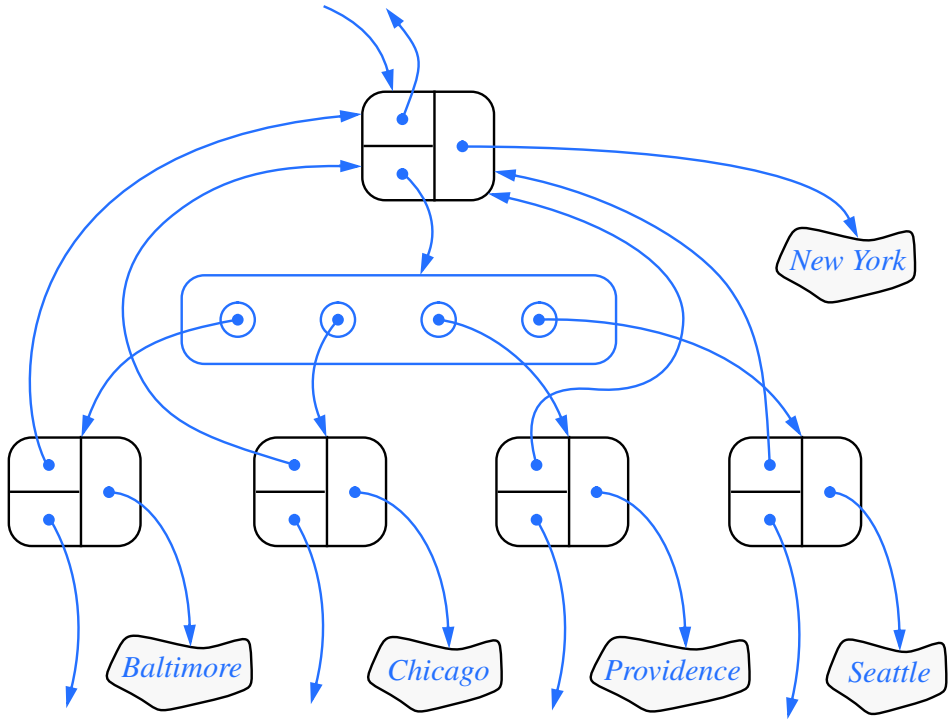
- save space
- update operations are costly (e.g. remove a node).

# Array based representation: pros and cons

- save space
- update operations are costly (e.g. remove a node).
- won't work for general trees

# Linked structure–general trees



# Tree traversal

- Pre-order

# Tree traversal

- Pre-order
- In-order

# Tree traversal

- Pre-order
- In-order
- Post-order

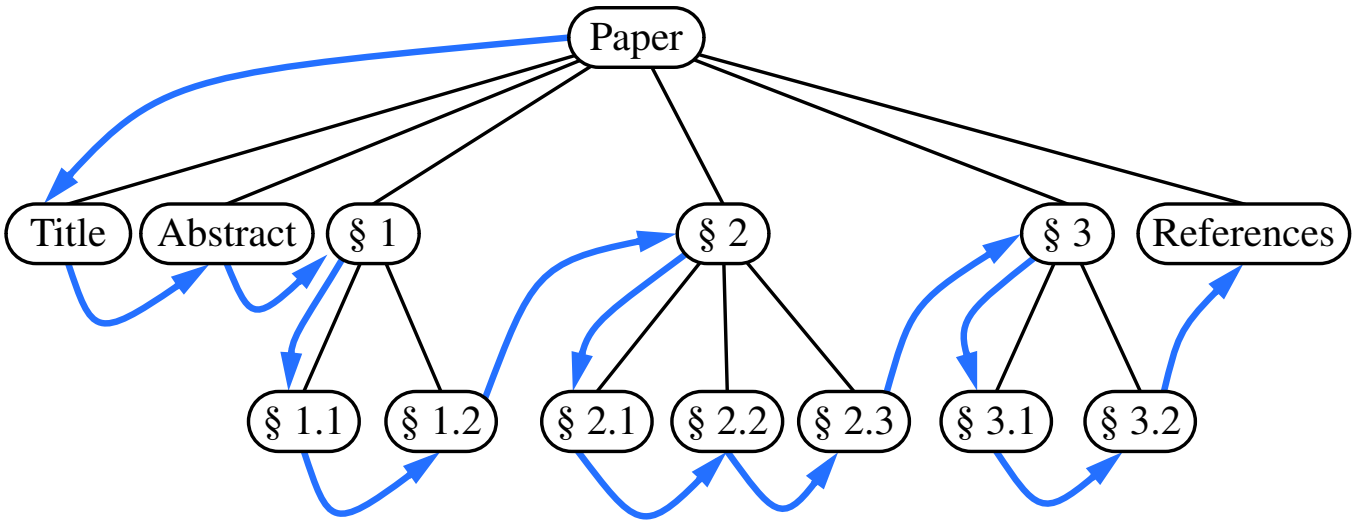
# Tree traversal

- Pre-order
- In-order
- Post-order
- Breadth-first

# Preorder traversal

**Algorithm 1:** PreOrder (Node n)

```
1 perform the visit action for node n
2 for child c of node n do
3 |   preorder(c)
4 end
```



# Application of pre-order traversal

```
Paper
Title
Abstract
§1
§1.1
§1.2
§2
§2.1
...
```
(a)

```
Paper
  Title
  Abstract
  §1
    §1.1
    §1.2
  §2
    §2.1
    ...
```
(b)

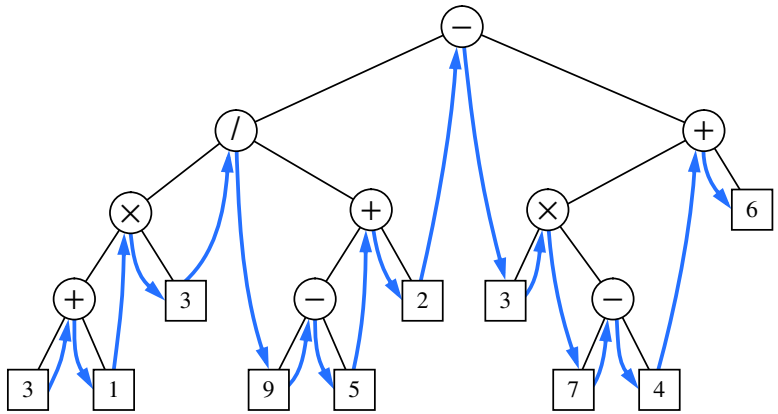
```
public<E>void printPreorder(Tree<E>T, Position<E> p){
  System.out.println(p.getElement());
  for (Position<E> c : T.children(p))
       printPreorderIndent(T, c);
  }

public<E>void printPreorderIndent(Tree<E>T, Position<E> p,int d){
  System.out.println(spaces(2*d) + p.getElement());
  for (Position<E> c : T.children(p))
       printPreorderIndent(T, c, d+1);
  }
```

# Inorder traversal (for binary tree only)

**Algorithm 2:** inOrder(Node n)

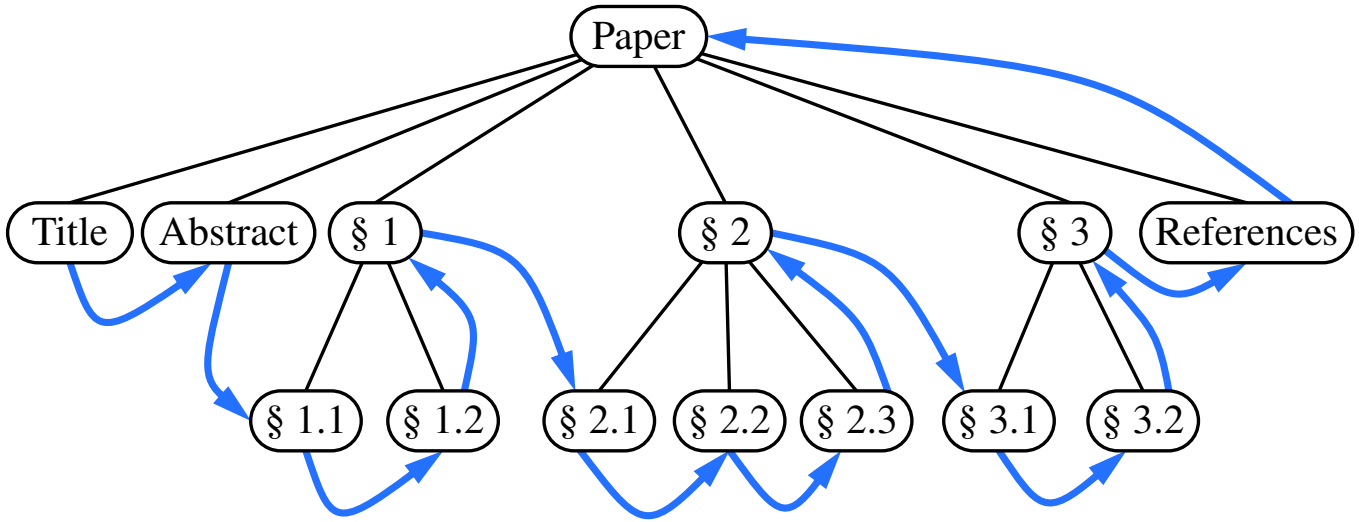
```
1 if n has a left child lc then
2 |   inOrder(lc);
3 end
4 perform the "visit" action for n;
5 if n has a right child rc then
6 |   inOrder(rc);
7 end
```



# Postorder traversal

**Algorithm 3:** PostOrder (Node n)

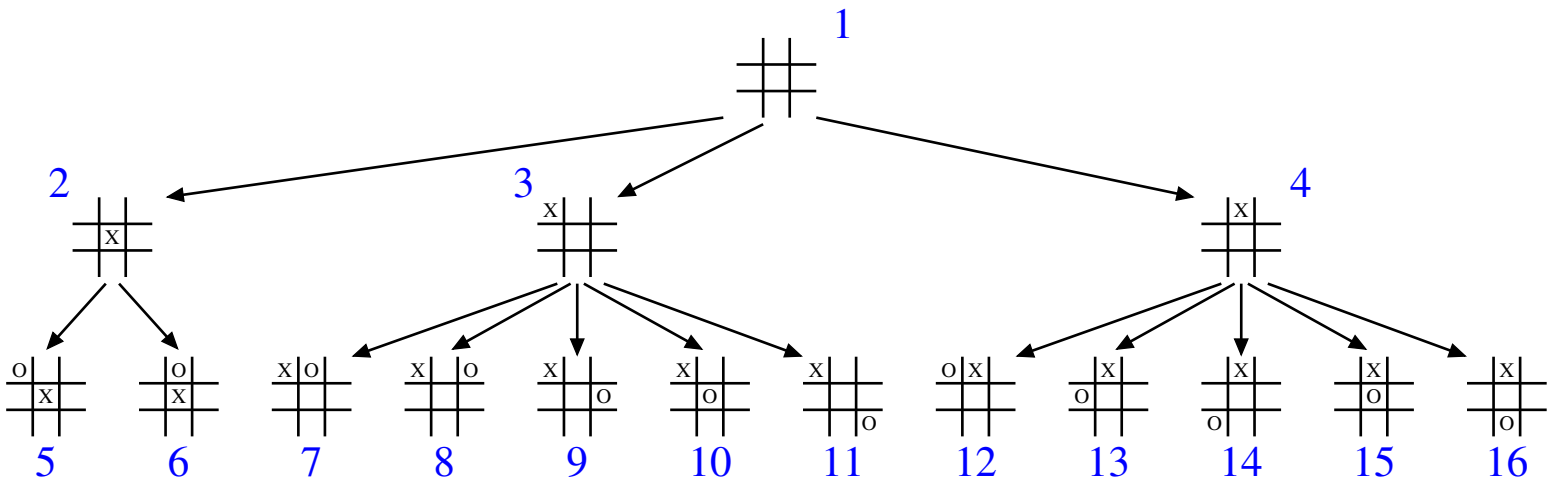
```
1 for child c of node n do
2 |   postorder(c)
3 end
4 perform the visit action for node n
```



# Breadth-first tree traversal

**Algorithm 4:** breadthFirst( )

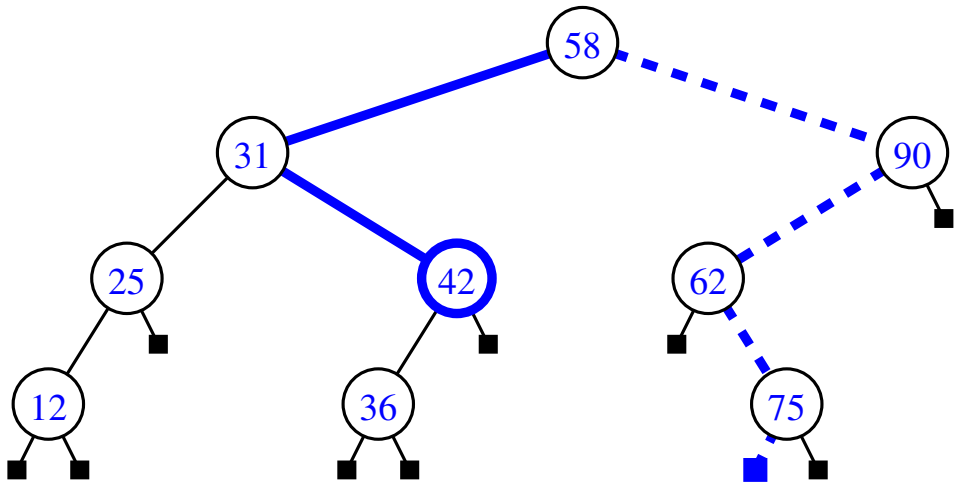
```
1 Initialize queue Q to contain root( );
2 while Q not empty do
3 |   n = Q.dequeue( );
4 |   perform the "visit" action for node n
5 |   for child c in children(n) do
6 |   |   Q.enqueue(c)
7 |   end
8 end
```



# Binary search tree

For each node n,

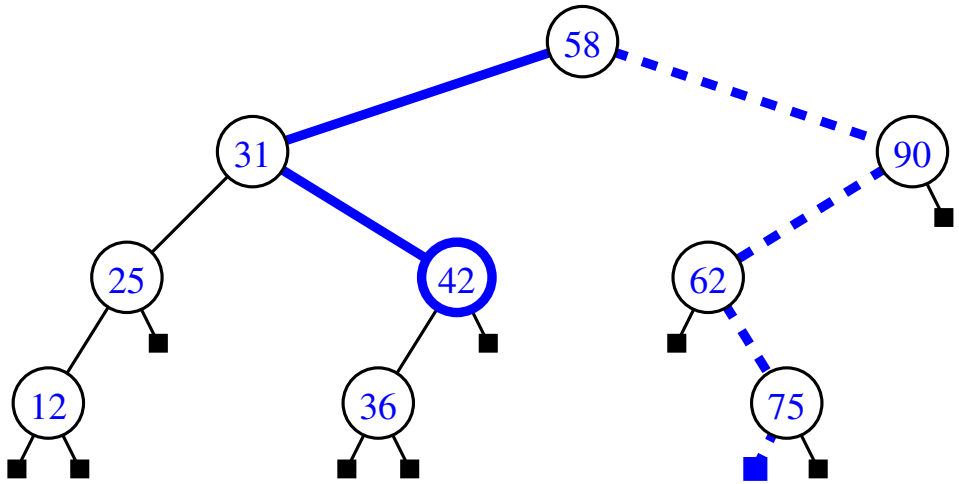
- Elements in the left subtree are less than n.



# Binary search tree

For each node n,

- Elements in the left subtree are less than n.
- Elements in the right subtree are greater than n.



# Takeaways

- Why trees
- Tree terminologies, depth, height, properties
  - Good and bad algorithms for tree height.
- Binary trees, proper/full binary tree, properties (maximum and minimum height of a binary tree),

$$h + 1 \leq n \leq 2^{h+1} - 1 \qquad \text{(for all binary trees)}$$

$$2h + 1 \leq n \leq 2^{h+1} - 1 \qquad \text{(for proper binary trees)}$$

$$n_E = n_I + 1 \qquad \text{(Internal and external nodes for proper binary trees)}$$

- Tree implementation
- Tree traversals (pre-order, in-order, post-order)
- Readings: Goodrich et al., P308-347